

Minimalni dominirajući skup

Darko Zorić

Sadržaj

1	Metod grube sile	3
2	Pohlepna pretraga	5
3	Lokalna pretraga	6
4	Simulirano kaljenje	10
5	Tabu pretraga	15
6	Genetski algoritam	18
7	Hibridni genetski algoritam (HGA-MDS)	25
8	Eksperimentalni rezultati i analiza performansi	31

Uvod

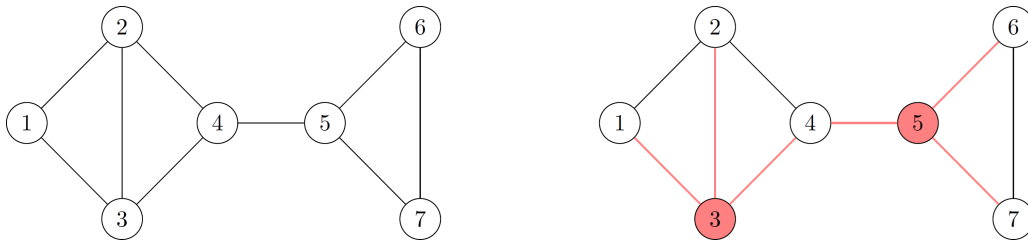
Dominirajući skup S je podskup čvorova takav da je svaki čvor grafa ili u samom skupu S ili je povezan sa nekim čvorom iz skupa S . Drugim rečima, svaki čvor u grafu je *dominiran* nekim čvorom iz skupa S .

Minimalni dominirajući skup (MDS) je dominirajući skup S sa najmanjim mogućim brojem čvorova (najmanjom kardinalnošću). Cilj problema je pronaći taj najmanji mogući skup. Kardinalnost tog skupa se obično se označava sa $\gamma(G)$.

Definicija. Dominirajući skup za graf G je skup $D \subseteq V(G)$ takav da je svaki čvor ili jedan od njegovih suseda sadržan u D , tj. za svako $v \in V(G)$ važi da je $v \in D$ ili postoji $u \in D$ sa $uv \in E(G)$.

Primera radi, zamislite mrežu gradova (čvorovi) i puteva (grane). Želite da postavite minimalan broj bolnica (čvorovi u S) tako da svaki grad (čvor) bude ili grad sa bolnicom, ili da mu je bolnica u susednom gradu.

Problem pronalaženja minimalnog dominirajućeg skupa pripada klasi NP-teških problema (engl. *NP-hard*). Problem odluke (da li postoji S veličine $\leq k$) je NP-kompletan, dok je problem optimizacije (pronaći S najmanje veličine) NP-težak.



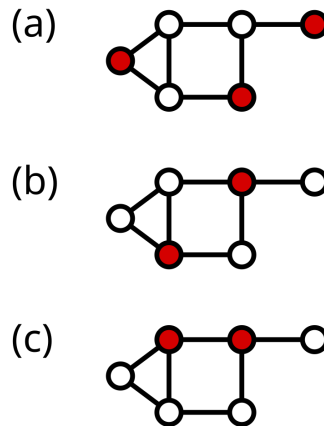
Slika 1: Graf sa odgovarajućim minimalnim dominirajućim skupom

Graf može imati više različitih podskupova čvorova iste, minimalne veličine, koji svi zadovoljavaju uslov dominacije. Drugim rečima, minimalni dominirajući skup ne mora biti jedinstven.

U ovom radu razmatrano je više različitih pristupa za rešavanje ovog problema. Korišćeni su:

- Metod grube sile
- Pohlepna pretraga
- Lokalna pretraga

- Simulirano kaljenje
- Tabu pretraga
- Genetski algoritam
- Modifikovani genetski algoritam



Slika 2: Graf sa različitim minimalnim dominirajućim skupovima

Prvih šest metoda implementirano je na osnovu gradiva sa kursa, dok je poslednji razvijen prema izabranom naučnom radu.

1 Metod grube sile

S obzirom na složenost problema, u mojoj implementaciji je kao prvi način rešavanja odabran **metod grube sile** (iscrpna pretraga), koji garantuje pronalazak egzaktnog, optimalnog rešenja. Ovaj pristup je primenljiv i praktičan samo za grafove male do umerene veličine.

Osnovna ideja ovog pristupa zasniva se na proveravanju svih mogućih podskupova čvorova u rastućem redosledu veličine, sve dok se ne pronađe prvi skup koji zadovoljava uslov dominacije.

Pošto se kandidati testiraju po veličini, prvi pronađeni dominirajući skup po definiciji mora biti minimalni dominirajući skup.

U pseudokodu, to izgleda ovako:

Algorithm 1 Metod grube sile

```
1: for  $k = 1$  do  $|V|$  do
2:   for svaki podskup  $S \subseteq V$  veličine  $k$  do
3:     if  $S$  je dominirajući skup then
4:       vrati  $S$  kao optimalno rešenje
5:     end if
6:   end for
7: end for
```

Implementacija

U implementaciji su definisane dve osnovne funkcije:

- `is_dominating_set(graph, candidate_nodes)` - proverava da li dati skup čvorova `candidate_nodes` dominira graf. Za svaki čvor `node` u grafu proverava da li:
 - sam pripada `candidate_nodes`, ili
 - ima bar jednog suseda u `candidate_nodes`.

Ako postoji bar jedan čvor koje nije pokriven, funkcija vraća `False`; u suprotnom, `True`.

- `calculate_dominating_set(graph)` - generiše sve moguće podskupove pomoću `itertools.combinations`. Petlja kreće od najmanjih skupova i vraća prvi koji zadovoljava uslov dominacije.

Složenost algoritma

Iako garantuje tačno rešenje, metod grube sile je veoma vremenski zahtevan i neefikasan.

- **Generisanje kombinacija:** Najsporiji deo je generisanje svih mogućih podskupova. Ukupan broj podskupova je:

$$\sum_{k=1}^{|V|} \binom{|V|}{k} = 2^{|V|} - 1$$

(broj svih nepraznih podskupova).

- **Provera dominacije:** Za svaki kandidatski skup, funkcija `is_dominating_set` proverava sva temena V i njihove susede E . Ova provera traje približno $O(|V| + |E|)$.

Kombinujući ove faktore, ukupna vremenska složenost algoritma grube sile je:

$$O(2^{|V|} \cdot (|V| + |E|))$$

Gde je $|V|$ broj temena, a $|E|$ broj grana. Zbog eksponencijalne zavisnosti od broja temena $2^{|V|}$, ovaj algoritam je nepraktičan za velike grafove (npr. sa više od 15 čvorova).

Prednosti i ograničenja

Prednosti	Ograničenja
Jednostavna implementacija	Eksponencijalna složenost
Garantuje tačno rešenje	Neprimenjiv za veće grafove
Koristan kao referenca za testiranje heuristika	Memorijski zahtev raste brzo sa veličinom grafa

Tabela 1: Prednosti i ograničenja metode grube sile

2 Pohlepna pretraga

Pohlepna pretraga predstavlja aproksimativni algoritam koji se koristi za brzo pronalaženje rešenja bliskih optimalnom. Za razliku od egzaktnog pristupa, ona ne garantuje pronalazak minimalnog dominirajućeg skupa, ali u praksi daje vrlo dobra rešenja u polinomijalnom vremenu.

Ideja algoritma

U svakom koraku algoritam bira čvor koji donosi najveću trenutnu korist, tj. čvor koji pokriva najveći broj trenutno nepokrivenih čvorova. Drugim rečima, algoritam gradi dominirajući skup D iterativno:

1. Na početku, svi čvorovi su nepokriveni.

2. U svakom koraku bira se čvor v koji dominira najveći broj trenutno nepokrivenih čvorova.
3. Izabrani čvor se dodaje u skup D , a svi čvorovi koje on pokriva označavaju se kao pokriveni.
4. Proces se ponavlja sve dok svi čvorovi ne budu pokriveni.

Po završetku, skup D predstavlja pohlepno aproksimiran dominirajući skup.

Zatvoreno susedstvo

U ovom algoritmu koristi se koncept zatvorenog susedstva:

$$N[v] = \{v\} \cup \{u \mid (v, u) \in E\}$$

Ono sadrži sam čvor v i sve njegove neposredne susede. Zatvoreno susedstvo omogućava tačno određivanje skupa čvorova kojim čvor v dominira i time precizno izračunavanje koliko je koristan u svakom koraku algoritma.

Algorithm 2 Pohlepni algoritam za MDS

```

1:  $D \leftarrow \emptyset$ 
2:  $uncovered \leftarrow V$ 
3: while  $uncovered \neq \emptyset$  do
4:   izaberi čvor  $v$  koji pokriva najviše čvorova iz  $uncovered$ 
5:    $D \leftarrow D \cup \{v\}$ 
6:    $uncovered \leftarrow uncovered \setminus N[v]$ 
7: end while
8: return  $D$ 

```

Prednosti i ograničenja

3 Lokalna pretraga

Lokalna pretraga je metaheuristički pristup koji se koristi za pronalaženje kvalitetnih (blizu optimalnih) rešenja za NP-teške probleme, kao što je problem minimalnog dominirajućeg skupa. Ideja algoritma je da se krene od nekog početnog rešenja i da se ono iterativno poboljšava malim lokalnim izmenama, sve dok se ne pronade bolje rešenje ili dok ne istekne zadati broj iteracija.

Za razliku od:

Prednosti	Ograničenja
Brza implementacija i polinomijalna složenost	Ne garantuje optimalno rešenje
Daje rešenja bliska optimumu za mnoge grafove	Kvalitet rešenja zavisi od strukture grafa
Lako se kombinuje sa drugim metaheuristikama	Može „zaglavljivati” u neoptimalnim rešenjima

Tabela 2: Prednosti i ograničenja pohlepne pretrage

- metoda grube sile, koji garantuje tačnost ali je eksponencijalno spor, i
- pohlepnog algoritma, koji je brz ali često suboptimalan,

lokalna pretraga balansira između brzine i kvaliteta rešenja, pa se često koristi kao jezgro mnogih metaheuristika (npr. simulirano kaljenje, tabu pretraga).

Ideja algoritma

Polazna tačka algoritma je inicijalno rešenje, koje se zatim modifikuje u koracima pomoću malih promena da bi se pronašla bolja konfiguracija. Svako novo rešenje se poredi sa trenutnim najboljim i zadržava se ako je bolje - po pravilu, ako sadrži manji broj čvorova u dominirajućem skupu.

Glavni elementi implementacije

Implementacija se sastoji iz tri osnovna dela:

1. Inicijalizacija rešenja (`create_initial_solution`) Algoritam može započeti pretragu sa dve različite strategije:

```

1 def create_initial_solution(graph, init_method="all",
  ↪ closed_neighbourhoods=None):
2     if init_method == "greedy":
3         # dodaj čvor koji pokriva najviše nepokrivenih
4         ...
5     return set(graph.keys()) # trivijalno rešenje

```

Način	Opis
"all"	Trivijalno rešenje: skup svih čvorova grafa $V(G)$. Ovo rešenje je uvek validno, ali daleko od optimalnog.
"greedy"	Pohlepno rešenje: primenjuje pohlepni pristup za izbor čvorova koji pokrivaju najveći broj nepokrivenih temena. Ovo daje znatno bolju početnu tačku za pretragu.

Tabela 3: Strategije inicijalizacije u lokalnoj pretrazi

2. Generisanje suseda (make_small_change) Ova funkcija proizvodi susedna rešenja tako što nasumično menja trenutno rešenje:

- **Uklanjanje (removal):** nasumično se bira čvor iz trenutnog rešenja i uklanja;
- **Zamena (swap):** nasumično se uklanja jedan čvor iz rešenja i dodaje drugi koji nije bio u rešenju.

Novo rešenje se prihvata samo ako i dalje dominira ceo graf, što se proverava funkcijom `is_dominating_set`.

```

1 if move_type == "removal":
2     node_to_remove = random.choice(tuple(solution_nodes))
3     ...
4 elif move_type == "swap":
5     node_to_remove = random.choice(tuple(solution_nodes))
6     node_to_add = random.choice(tuple(all_nodes - candidate_solution))

```

Kriterijum validnosti: Nova konfiguracija (nakon uklanjanja ili zamene) se prihvata samo ako ostane validan Dominirajući Skup (provereno pomoću funkcije `is_dominating_set`). Ako promena naruši date uslove, vraća se originalno rešenje.

3. Glavna petlja (local_search) Glavni deo algoritma iterira zadati broj puta (`max_iters`):

1. Generiše se novi kandidat pomoću `make_small_change`;
2. Proverava se njegova veličina (broj čvorova u dominirajućem skupu);
3. Ako je kandidat bolji (manji skup), ažurira se trenutno najbolje rešenje.

```

1  for _ in range(max_iters):
2      new_solution_nodes = make_small_change(solution_nodes, graph,
        ↪ all_nodes, closed_neighbourhoods)
3      if len(new_solution_nodes) < best_solution_size:
4          best_solution_nodes = new_solution_nodes
5          best_solution_size = len(new_solution_nodes)

```

Algorithm 3 Lokalna pretraga za MDS

```

1: Inicijalizuj rešenje  $S_0$  (trivijalno ili pohlepno)
2:  $S^* \leftarrow S_0$ 
3: for  $i = 1$  to  $max\_iters$  do
4:    $S' \leftarrow \text{napravi\_malu\_promenu}(S)$ 
5:   if  $S'$  je validan i bolji od  $S^*$  then
6:      $S^* \leftarrow S'$ 
7:   end if
8:    $S \leftarrow S'$ 
9: end for
10: return  $S^*$ 

```

Kriterijum validnosti

Validnost svakog rešenja proverava se funkcijom `is_dominating_set`, koja proverava da li svaki čvor u grafu:

- ili pripada trenutnom skupu rešenja, ili
- ima suseda u tom skupu.

Za proveru se koristi zatvoreno susedstvo:

$$N[v] = \{v\} \cup \{u \mid (v, u) \in E\}$$

Parametri algoritma

Parametar	Opis	Tipična vrednost	Obrazloženje
max_iters	Maksimalan broj iteracija	500–1000	Određuje trajanje pretrage i balans između kvaliteta i vremena izvršavanja
init_method	Način inicijalizacije rešenja ("all" ili "greedy")	"greedy"	Pohlepno rešenje daje bržu konvergenciju
move_type	Tip poteza u generisanju suseda	removal / swap	Obezbeđuje lokalnu eksploraciju prostora rešenja

Tabela 4: Parametri lokalne pretrage

Prednosti i ograničenja

Prednosti	Ograničenja
Brz i jednostavan za implementaciju	Može „zaglavljivati” u lokalnim minimumima
Dobro balansira između kvaliteta i vremena	Potrebno fino podešavanje broja iteracija
Pogodan za kombinovanje sa drugim metaheuristikama	Rezultat zavisi od početnog rešenja

Tabela 5: Prednosti i ograničenja metode lokalne pretrage

4 Simulirano kaljenje

Simulirano kaljenje (engl. *Simulated Annealing* - *SA*) je sofisticirana metaheuristika inspirisana procesom kaljenja metala u metalurgiji. U fizici, kaljenje je proces zagrevanja i kontrolisanog hlađenja materijala kako bi se postiglo stanje minimalne energije (najstabilnije i najjače stanje).

U kontekstu optimizacije MDS problema, ovaj algoritam neprekidno istražuje prostor rešenja kako bi pronašao minimalni dominirajući skup. Ključna prednost SA u odnosu na standardnu lokalnu pretragu je njegova sposobnost da privremeno prihvati lošija rešenja, čime izbegava zaglavljivanje u lokalnim minimumima i povećava šansu da se pronađe globalno optimalno ili skoro optimalno rešenje.

Za razliku od prethodnih algoritama koji rade isključivo sa validnim dominirajućim skupovima, simulirano kaljenje koristi funkciju koja izračunava cenu rešenja (`calculate_cost`) i koja omogućava rad sa nevalidnim rešenjima.

Funkcija cene

Formula za izračunavanje cene rešenja data je kao:

$$\text{Cena} = |D| + (\text{Nepokriveni čvorovi} \cdot \text{Težina kazne})$$

Gde su:

- **Veličina skupa** ($|D|$): broj čvorova u trenutnom dominirajućem skupu (primarni cilj minimizacije);
- **Težina kazne** (PENALTY): fiksna visoka vrednost (u ovom kodu, `num_nodes + 1`), koja osigurava da je svako nevalidno rešenje (gde ima nepokrivenih čvorova) uvek skuplje od bilo kog validnog rešenja (gde je broj nepokrivenih = 0).

Time se algoritam usmerava ka pronalaženju rešenja koje ima mali $|D|$, ali mu se dozvoljava da prolazi kroz nevažeća stanja ($\text{Cena} > |D|$) radi efikasnijeg istraživanja prostora rešenja.

Glavni mehanizmi simuliranog kaljenja

Glavni mehanizam (`simulated_annealing` funkcija) zasniva se na promenljivoj temperaturi (T) i kriterijumu prihvatanja novog rešenja.

Inicijalizacija i hlađenje

- **Inicijalna temperatura** (T_0): algoritam kreće sa visokom temperaturom (npr. $T_0 = 10.0$). Na visokoj T , verovatnoća prihvatanja lošijih rešenja je velika, što proširuje prostor pretrage.

- **Hlađenje:** u svakoj iteraciji, temperatura se smanjuje množenjem sa faktorom hlađenja α (npr. 0.999), što predstavlja geometrijsku strategiju hlađenja. Hlađenje se nastavlja dok se ne postigne minimalna temperatura ($T_{\min}$). Sa padom T , smanjuje se i verovatnoća prihvatanja lošijih rešenja.

Kriterijum prihvatanja

Nakon generisanja novog rešenja D_{novi} (kandidata) i izračunavanja promene troška

$$\Delta = \text{Trošak}(D_{\text{novi}}) - \text{Trošak}(D_{\text{trenutni}}),$$

primenjuje se tzv. *Metropolisov kriterijum*:

1. **Poboljšanje** ($\Delta \leq 0$): ako je novo rešenje bolje ili jednako trenutnom, ono se uvek prihvata.
2. **Pogoršanje** ($\Delta > 0$): ako je novo rešenje lošije (skuplje), ono se prihvata sa određenom verovatnoćom:

$$\text{Verovatnoća prihvatanja} = e^{-\Delta/T}$$

Ova verovatnoća zavisi od:

- Δ (razlike cene): što je pogoršanje veće, verovatnoća prihvatanja je manja;
- T (temperatura): što je temperatura viša, verovatnoća prihvatanja lošeg rešenja je veća.

Generisanje suseda (make_small_change)

Novi kandidati se generišu pomoću dve vrste poteza, prikazane u tabeli 6.

Tip poteza	Opis	Verovatnoća
multi_flip	Nasumično uključivanje/isključivanje više čvorova u rešenju	0.9
swap	Klasična zamena jednog čvora u skupu sa jednim koji nije u skupu	0.1

Tabela 6: Vrste poteza pri generisanju suseda u SA algoritmu

Broj prebacivanja (`num_flips`) u potezu `multi_flip` zavisi od veličine grafa i trenutne temperature, što znači da su skokovi veći na početku (kada je T visoka), a postepeno se smanjuju tokom hlađenja.

Izbor poteza implementiran je kao:

```
move_type = random.choices(["multi_flip", "šwap"], weights=[0.9, 0.1])[0]
```

Inicijalno rešenje (`create_initial_solution`)

Iako SA može započeti od bilo kog početnog stanja, ova implementacija koristi *pohlepni pristup sa nasumičnim izborom* - biranjem između tri najbolja čvora (`top_k` = 3), kako bi se dobio relativno dobar početni skup, ali sa dozom nasumičnosti koja obezbeđuje veću diverzifikaciju i izbegavanje rane konvergencije.

Popravka rešenja (`repair_solution`)

Zbog upotrebe funkcije cene koja dozvoljava nevalidne skupove, na samom kraju rada SA algoritma poziva se funkcija `repair_solution` kako bi se osiguralo da je finalno rešenje zaista validan dominirajući skup.

Ako finalno rešenje nije dominantno, algoritam ga popravljja korišćenjem jednostavnog pohlepnog pristupa - dodavanjem minimalnog potrebnog broja čvorova kako bi svi čvorovi u grafu bili pokriveni.

Algorithm 4 Simulirano kaljenje za MDS

- 1: Inicijalizuj $T = T_0$, generiši početno rešenje D
 - 2: **while** $T > T_{min}$ **do**
 - 3: generiši susedno rešenje D'
 - 4: izračunaj $\Delta = C(D') - C(D)$
 - 5: **if** $\Delta \leq 0$ **then** prihvati D'
 - 6: **else** prihvati sa verovatnoćom $e^{-\Delta/T}$
 - 7: **end if**
 - 8: $T \leftarrow \alpha \cdot T$
 - 9: **end while**
-

Parametri algoritma

Parametar	Opis	Tipična vrednost	Obrazloženje
T0	Početna temperatura	10.0	Velika temperatura omogućava širu pretragu i prihvatanje lošijih rešenja na početku
alpha	Faktor hlađenja	0.999	Kontroliše brzinu hlađenja; vrednosti bliže 1 znače sporije hlađenje
T_min	Minimalna temperatura	1×10^{-6}	Kriterijum zaustavljanja — kada više nema značajnih promena u rešenju
max_iters	Maksimalan broj iteracija	10^4 – 10^5	Duža pretraga daje stabilnije rezultate
PENALTY	Kazna za nevalidno rešenje	num_nodes + 1	Osigurava da svako nevalidno rešenje ima veću cenu od bilo kog validnog
top_k	Broj kandidata u inicijalnoj fazi	3	Kombinuje kvalitet i diverzifikaciju u početnoj fazi
move_type	Tip promene u generisanju suseda	multi_flip / swap	Veća šansa za multi_flip u ranim fazama algoritma

Tabela 7: Parametri algoritma simuliranog kaljenja

Prednosti i ograničenja

Prednosti	Ograničenja
Efikasno izbegava lokalne minimume	Parametri zahtevaju fino podešavanje
Može raditi i sa nevalidnim rešenjima	Veće vreme izvršavanja u odnosu na pohlepni pristup
Fleksibilan i robustan metod	Kvalitet zavisi od funkcije hlađenja i početne temperature

Tabela 8: Prednosti i ograničenja metode simuliranog kaljenja

5 Tabu pretraga

Tabu pretraga je metaheuristika za globalnu optimizaciju koja koristi mehanizme pamćenja da bi prevazišla lokalne minimume i istražila širi prostor mogućih rešenja. Ona se zasniva na principu lokalne pretrage, ali uvodi dodatne strukture podataka koje sprečavaju vraćanje na već posećene konfiguracije ili ponavljanje istih poteza.

Osnovna ideja je da se, u svakoj iteraciji, napravi mali pomak (potez) iz trenutnog rešenja prema nekom susednom rešenju. Za razliku od obične lokalne pretrage, tabu pretraga dozvoljava i poteze koji privremeno kvare rešenje (povećavaju vrednost funkcije cilja), ako se time omogućava izlazak iz lokalnog optimuma.

Da bi se izbeglo besciljno vraćanje u ranije stanja, koristi se **tabu lista**, koja zabranjuje ponavljanje određenih poteza tokom ograničenog broja iteracija.

Glavni mehanizmi tabu pretrage

Tabu pretraga se oslanja na tri glavna mehanizma: tabu listu, kriterijum aspiracije i dugoročnu memoriju.

Tabu lista (Tabu list)

Tabu lista predstavlja kratkoročnu memoriju koja čuva upravo izvršene poteze i zabranjuje njihovo ponavljanje u narednih nekoliko iteracija. Na taj način se algoritam usmerava da istražuje nove oblasti prostora rešenja.

U implementaciji se koristi kao rečnik:

```
tabu_list: Dict[Tuple, int]
```

gde je ključ konkretan potez (npr. ('swap', 3, 7)), a vrednost broj iteracije do koje potez ostaje zabranjen (`it + tabu_tenure`).

Parametar `tabu_tenure` određuje koliko dugo potez ostaje tabu. Ako je `tabu_tenure` prevelik, algoritam gubi fleksibilnost; ako je premali, postoji rizik vraćanja u prethodno rešenje. U ovoj implementaciji koristi se umerena vrednost (npr. 7-10 iteracija), što omogućava dobar balans između *intenzifikacije* (lokalno poboljšanje) i *diverzifikacije* (istraživanje novih područja).

Kriterijum aspiracije (Aspiration criterion)

Kriterijum aspiracije je pravilo koje dozvoljava tabu potez ako vodi ka boljem rešenju od trenutnog globalnog optimuma. Drugim rečima, zabrana iz tabu liste se može *nadjačati* ako je potez izuzetno koristan.

U implementaciji se to sprovodi sledećim uslovom:

```
if is_tabu and sol_value >= global_best_value:
    continue
```

Što znači:

- Ako je potez označen kao tabu, on se preskače;
- Ali ako taj potez proizvodi bolje rešenje (manji broj čvorova u dominirajućem skupu), tabu zabrana se ignoriše.

Ovaj mehanizam omogućava algoritmu da ne propusti priliku za poboljšanje, čak i ako potez tehnički spada u „zabranjene“.

Dugoročna memorija (Long-Term Memory)

Dugoročna memorija predstavlja statističko pamćenje koje beleži koliko je puta određeni potez primenjen tokom pretrage.

U kodu je realizovana na sledeći način:

```
long_term_memory[chosen_move] = long_term_memory.get(chosen_move, 0) + 1
```

Ove informacije mogu poslužiti u fazama diverzifikacije, gde se prioritet daje ređim, manje korišćenim potezima - čime se podstiče istraživanje novih delova prostora rešenja.

Algorithm 5 Tabu pretraga za MDS problem

```

1: Inicijalizuj početno rešenje  $D$  (svi čvorovi uključeni)
2: Postavi  $best\_solution \leftarrow D$  i  $best\_value \leftarrow cost(D)$ 
3: for svaku iteraciju  $t = 1$  do  $max\_iters$  do
4:   Generiši skup mogućih poteza: removal / swap
5:   Filtriraj tabu poteze (osim onih koji zadovoljavaju aspiracioni kriterijum)
6:   Odaberi najboljeg kandidata  $D'$  iz dozvoljenih poteza
7:   Primeni odabrani potez i ažuriraj pokrivenost grafa
8:   if  $cost(D') < best\_value$  then
9:      $best\_solution \leftarrow D'$ 
10:     $best\_value \leftarrow cost(D')$ 
11:   end if
12:   Dodaj odabrani potez u Tabu listu sa trajanjem  $tabu\_tenure$ 
13: end for
14: Vrati  $best\_solution$  i  $best\_value$ 

```

Parametri algoritma

Parametar	Opis	Tipična vrednost	Obrazloženje
max_iters	Maksimalan broj iteracija	500–1000	Veći broj omogućava dublju pretragu, ali povećava vreme izvođenja
tabu_tenure	Broj iteracija tokom kojih potez ostaje tabu	7-10	Obezbeđuje balans između istraživanja i izbegavanja ciklusa
swap_cap	Maksimalan broj kandidata za zamenu u jednoj iteraciji	64-256	Ograničava broj suseda i ubrzava pretragu
long_term_memory	Statističko praćenje učestalosti poteza	-	Služi za diversifikaciju rešenja tokom pretrage

Tabela 9: Parametri algoritma Tabu pretrage

6 Genetski algoritam

Genetski algoritam (GA) je metaheuristička tehnika inspirisana procesima evolucije i prirodne selekcije. Zasniva se na ideji da se potencijalna rešenja problema posmatraju kao “jedinke” u populaciji koje se tokom iteracija reprodukuju, kombinuju i mutiraju, pri čemu preživljavaju one koje imaju najbolju prilagođenost (fitness).

Tokom evolucije, algoritam kombinuje dobre osobine postojećih rešenja (ukrštanje) i unosi kontrolisane varijacije (mutacija), uz povremenu popravku rešenja koja nisu validna.

Reprezentacija jedinke

Svaka jedinka predstavlja binarnu vektorsku reprezentaciju rešenja:

$$\text{genetic_code} = [g_1, g_2, \dots, g_n],$$

gde je:

$$g_i = \begin{cases} 1 & \text{ako je \u010dvor deo dominiraju\u0107eg skupa } D, \\ 0 & \text{ako nije.} \end{cases}$$

U implementaciji se koristi numpy niz tipa `uint8`, \u0161to omogućava brze bit-operacije i manipulaciju velikim populacijama.

Struktura klase Individual

Klasa `Individual` sadr\u017ei sve informacije o jedinki i njenim osobinama:

- `genetic_code` – binarna reprezentacija rešenja
- `fitness` – vrednost funkcije prilago\u0107enosti
- `solution_size` – broj izabranih \u010dvorova
- `num_uncovered_nodes` – broj nepokrivenih \u010dvorova
- `closed_neighbourhoods` / `closed_masks` – unapred pripremljene strukture za brzo ra\u010dunanje dominacije pomo\u0107u bit-maski

Funkcija prilago\u0107enosti (fitness)

Za svaku jedinku I , prilago\u0107enost se ra\u010duna kao:

$$\text{fitness}(I) = (\text{broj pokrivenih \u010dvorova}) \cdot (|V| + 1) - |D|,$$

gde:

- $|V| + 1$ je te\u017eina koja favorizuje potpuno pokrivanje grafa,
- $|D|$ penalizuje ve\u0107e skupove.

Na ovaj na\u010din validna i manja rešenja dobijaju ve\u0107u fitness vrednost, dok nevalidna automatski gube prednost.

Korišćenje bit-mask strukture (closed_masks)

Bit-mask reprezentacija značajno ubrzava izračunavanja pokrivenosti čvorova i fitness funkcije. Za svaki čvor v u grafu definišemo njegovo zatvoreno susedstvo $N[v]$:

$$N[v] = \{v\} \cup \{u \mid (v, u) \in E\}.$$

Svako zatvoreno susedstvo se pretvara u celobrojnu masku čiji binarni zapis označava koje čvorove taj čvor pokriva.

Primer: Ako graf ima 5 čvorova, i za čvor $v = 2$:

$$N[2] = \{1, 2, 4\} \Rightarrow \text{mask}(2) = 0b10110 = 22$$

```
1 def calculate_mask_of_covered_nodes(self) -> int:
2     mask_covered = 0
3     for u in np.flatnonzero(self.genetic_code):
4         mask_covered |= self.closed_masks[u]
5     return mask_covered
```

Za svaki čvor koji je deo rešenja ($\text{gen} = 1$), uzima se njegova maska i kombinuje pomoću bitwise OR operatora. Nepokriveni čvorovi se izračunavaju pomoću XOR operacije nad svim čvorovima (mask_all) i trenutno pokrivenim čvorovima (mask_covered):

$$\text{mask_uncovered} = \text{mask_all} \oplus \text{mask_covered}$$

Glavni delovi implementacije

1. Inicijalizacija populacije Početna populacija se generiše nasumično, pri čemu se oko 5% čvorova postavlja na 1, a zatim se primenjuje funkcija `repair()` kako bi jedinke postale validni dominirajući skupovi.

```
1 def initialize_population(population_size, closed_neighbourhoods,
2     ↪ closed_masks):
3     population = []
4     for _ in range(population_size):
5         individual = Individual(closed_neighbourhoods, closed_masks)
6         individual.repair()
7         population.append(individual)
8     return population
```

2. Selekcija roditelja Koristi se turnirska selekcija (*tournament selection*):

- Slučajno se bira nekoliko jedinki.
- Jedinka sa najvećim fitness-om postaje roditelj.

```
1 def tournament_selection(population, tournament_size):
2     contestants = random.sample(population, k=tournament_size)
3     return max(contestants, key=lambda ind: ind.fitness)
```

Ovaj pristup održava balans između eksploatacije (favorizovanje boljih rešenja) i diverzifikacije (nasumičnost izbora).

3. Ukrštanje (Crossover) Roditelji se kombinuju uniformnim ukrštanjem:

- Svaki gen se nasleđuje od jednog od roditelja sa verovatnoćom $p = 0.5$.

```
1 mask = (np.random.rand(parent1.num_nodes) < 0.5)
2 c1 = np.where(mask, parent1.genetic_code, parent2.genetic_code)
3 c2 = np.where(mask, parent2.genetic_code, parent1.genetic_code)
```

Ovo prenosi dobre osobine oba roditelja i održava genetsku raznovrsnost.

4. Mutacija Mutacija nasumično menja vrednost jednog gena (flip $0 \rightarrow 1$ ili $1 \rightarrow 0$) sa malom verovatnoćom `mutation_rate` (npr. 0.03). Ako mutacijom nastane nevalidna jedinka, primenjuje se `repair()`.

```
1 if random.random() < mutation_rate:
2     index = random.randrange(individual.num_nodes)
3     individual.genetic_code[index] ^= 1
4     ...
```

5. Popravka jedinki (Repair) Ova funkcija osigurava da svaka jedinka bude validan dominirajući skup:

1. Identifikuju se nepokriveni čvorovi pomoću bit-maski.
2. Dodaju se čvorovi koji pokrivaju najviše nepokrivenih čvorova.

3. Nakon pokrivanja, pokušava se ukloniti suvišne čvorove koji ne utiču na pokrivanje.

Ovo sprečava gomilanje nevalidnih genoma u jedinkama populacije.

6. Elitizam Na svakoj iteraciji se određeni broj najboljih jedinki direktno prenosi u sledeću generaciju, što sprečava gubitak kvalitetnih rešenja:

```
elites = heapq.nlargest(elitism_size, population, key=lambda ind: ind.fitness)
```

7. Adaptivna verovatnoća popravke (p_repair)

Verovatnoća da će se deca automatski popravljati raste tokom evolucije:

$$p_{\text{repair}}(it) = p_{\text{start}} + (p_{\text{end}} - p_{\text{start}}) \cdot \frac{it}{\text{max_iters}}$$

- **p_start:** Početna verovatnoća popravke (npr. 0.30). Na početku evolucije, visoka nasumičnost je poželjna, pa popravka treba da se primenjuje ređe.
- **p_end:** Krajnja verovatnoća popravke (npr. 1.00). Kasnije se popravka primenjuje skoro uvek, čime se smanjuje broj nevalidnih rešenja.
- **it:** Tekuća iteracija (generacija).
- **max_iters:** Ukupan broj generacija, koristi se za normalizaciju.

Na početku algoritma se favorizuje diverzifikacija, a kasnije eksploatacija postojećih rešenja.

Algorithm 6 Genetski algoritam za problem minimalnog dominirajućeg skupa

```
1: Inicijalizuj populaciju  $P$  veličine population_size
2: Izračunaj fitness za svaku jedinku u  $P$ 
3: for svaku generaciju  $it = 1$  do max_iters do
4:   Sačuvaj elitism_size najboljih jedinki (elite)
5:   nova_populacija = elite
6:   while  $|nova\_populacija| < |P|$  do
7:     roditelj1 = turnir_selekcija(P, tournament_size)
8:     roditelj2 = turnir_selekcija(P, tournament_size)
9:     (dete1, dete2) = uniformno_ukrštanje(roditelj1, roditelj2)
10:    mutiraj(dete1, mutation_rate), mutiraj(dete2, mutation_rate)
11:    Sa verovatnoćom  $p_{\text{repair}}(it)$  popravi dete1 i dete2
12:    Dodaj dete1 i dete2 u novu_populaciju
13:   end while
14:    $P = nova\_populacija$ 
15: end for
16: Vrati najbolju jedinku iz  $P$ 
```

Parametri algoritma

Parametar	Opis	Tipična vrednost	Obrazloženje
population_size	Broj jedinki u populaciji	200-400	Veća populacija povećava diverzifikaciju i smanjuje rizik od lokalnih minimuma
tournament_size	Broj jedinki u turniru	3-7	Veći turnir pojačava selekcionu pritisak na bolje jedinke
mutation_rate	Verovatnoća mutacije pojedinačnog gena	0.02-0.05	Umerena mutacija sprečava preranu konvergenciju i održava raznovrsnost
elitism_size	Broj jedinki koje se prenose nepromenjene	5-15	Obezbeđuje očuvanje najboljih rešenja kroz generacije
max_iters	Maksimalan broj generacija	1000-2000	Više iteracija omogućava finije poboljšanje rešenja
p_repair_start	Početna verovatnoća popravke	0.30	Rane generacije dozvoljavaju više varijacija, manje popravki
p_repair_end	Završna verovatnoća popravke	1.00	Kasnije generacije skoro uvek popravljaju decu za stabilnija rešenja

Prednosti i ograničenja

Prednosti	Ograničenja
Sposoban da pronalazi vrlo kvalitetna (često optimalna) rešenja	Zahteva pažljivo podešavanje parametara
Kombinuje globalnu i lokalnu pretragu (eksploatacija + eksploracija)	Veće vreme izvršavanja u odnosu na lokalne heuristike
Održava diverzitet populacije i izbegava lokalne minimume	Potrebno više memorije zbog populacije
Fleksibilan - lako se kombinuje sa drugim metaheuristikama	Stohastička priroda - različiti rezultati pri različitim pokretanjima

7 Hibridni genetski algoritam (HGA-MDS)

Hibridni genetski algoritam (HGA-MDS) predstavlja naprednu varijantu standardnog genetskog algoritma, posebno prilagođenu rešavanju problema minimalnog dominirajućeg skupa. Ovaj pristup je prvi put predstavljen u radu **Abdel-Rahman Hedara i Rashada Ismaila (2010)**, gde je pokazano da kombinacija genetskog evolutivnog procesa sa lokalnim pretragama i mehanizmima intenzifikacije dovodi do znatno boljih rezultata od standardnog GA.

HGA-MDS kombinuje:

- genetsku evoluciju (ukrštanje, mutacija, selekcija),
- lokalne i globalne pretrage (*intensification mechanisms*),
- rang-baziranu selekciju (*linear ranking*) za stabilnu kontrolu konvergencije,
- bitarray strukturu za izuzetno brzu binarnu reprezentaciju rešenja.

Reprezentacija jedinki - bitarray struktura

Za razliku od standardne implementacije koja koristi `numpy` nizove, HGA-MDS koristi biblioteku `bitarray`, koja nije deo standardnog Python jezika, ali omogućava memorijski efikasnu i brzu bitovnu manipulaciju. Svaka jedinka (rešenje) predstavlja binarni niz:

$$\mathbf{x} = [x_1, x_2, \dots, x_n], \quad x_i \in \{0, 1\},$$

gde $x_i = 1$ znači da je čvor uključen u dominirajući skup.

Pored same genetske reprezentacije, za svaki čvor grafa se izračunava i bit-niz zatvorenog susedstva:

```
1 def build_bit_neighborhoods(graph):
2     for i in range(num_nodes):
3         b = bytearray(num_nodes)
4         b.setall(0)
5         b[i] = 1
6         for v in graph[i]:
7             b[v] = 1
8         neighborhoods.append(b)
```

Ovakav pristup omogućava da se proveravanje pokrivenosti čvorova radi pomoću brzih bitovnih OR operacija, umesto skupovnih operacija nad listama. Time je svaka evaluacija značajno ubrzana.

Funkcija prilagođenosti (Fitness Function)

HGA-MDS koristi specijalno definisanu funkciju prilagođenosti predloženu u radu Hedara i Ismaila:

$$fitness(\mathbf{x}) = \frac{n}{|V|} + \frac{1}{|V| \cdot \gamma_x(G)}$$

gde je:

- n – broj čvorova pokrivenih rešenjem $\mathbf{x}$,
- $\gamma_x(G)$ – broj čvorova koji pripadaju samom rešenju (veličina skupa),
- $|V|$ – ukupan broj čvorova u grafu.

Prvi član $\frac{n}{|V|}$ meri pokrivenost grafa (idealno 1.0 ako je rešenje validno), dok drugi član $\frac{1}{|V|\gamma_x(G)}$ penalizuje prevelike skupove i nagrađuje manja, efikasnija rešenja.

Linear Ranking Selection (Rang-bazirana selekcija)

Standardna rulet-selekcija u genetskim algoritmima može izazvati prebrzu konvergenciju jer jedinke sa visokim fitness-om brzo dominiraju populacijom. Da bi se to sprečilo, koristi se *ranking selekcija* - uvedena u radu Bakera (1985).

Umesto da se verovatnoća odabira zasniva direktno na apsolutnom fitnessu, ona se zasniva na rangju jedinice u populaciji.

Princip rada

1. Populacija se sortira po fitnessu (najbolji = rang 1).
2. Svakom rangju se dodeljuje očekivani broj potomaka između minimalne i maksimalne vrednosti.
3. Linearna funkcija rangova određuje verovatnoću selekcije:

$$E(i) = \text{MAX} - (i - 1) \cdot \text{INC}$$

gde je

$$\text{INC} = \frac{2(\text{MAX} - 1.0)}{N - 1}, \quad \text{MIN} = 2.0 - \text{MAX}$$

Parametar MAX kontroliše selekcijski pritisak:

- MAX = 2.0 → brza konvergencija (75% populacije aktivno),
- MAX = 1.1 → stabilna evolucija (95% populacije aktivno).

Ovaj mehanizam osigurava da:

- verovatnoća selekcije monotonno raste sa boljim rangom,
- zbir svih verovatnoća ostaje jednak veličini populacije.

Primer

Pretpostavimo populaciju od $N = 5$ jedinice sa sledećim vrednostima fitnessa:

Jedinka	Fitness	Rang (1 = najbolji)
A	0.92	1
B	0.84	2
C	0.73	3
D	0.62	4
E	0.51	5

Koristeći formulu:

$$E(i) = \text{MAX} - (i - 1) \cdot \text{INC}, \quad \text{INC} = \frac{2(\text{MAX} - 1.0)}{N - 1}$$

Za MAX = 1.1 i N = 5:

$$\text{INC} = \frac{2(1.1 - 1.0)}{4} = 0.05$$

Rang (i)	Jedinka	$E(i) = 1.1 - (i - 1) \cdot 0.05$	$E(i)$
1	A	1.1 - 0	1.10
2	B	1.1 - 0.05	1.05
3	C	1.1 - 0.10	1.00
4	D	1.1 - 0.15	0.95
5	E	1.1 - 0.20	0.90

Verovatnoće odabira za *mating pool* dobijamo normalizacijom:

$$P(i) = \frac{E(i)}{\sum E(i)}, \quad \sum E(i) = 1.10 + 1.05 + 1.00 + 0.95 + 0.90 = 5.00$$

Jedinka	$E(i)$	$P(i) = E(i)/5.0$	Verovatnoća izbora
A	1.10	0.22	22%
B	1.05	0.21	21%
C	1.00	0.20	20%
D	0.95	0.19	19%
E	0.90	0.18	18%

Razlika između najboljeg (A) i najlošijeg (E) je mala (22% vs 18%), što znači da sve jedinke imaju šansu da učestvuju u evoluciji, čime se sprečava prerana konvergencija.

Intenzifikacione faze (Local Search, Filtering, Elite Inspiration)

Pored standardnih genetskih operatora (ukrštanje, mutacija), HGA-MDS koristi tri faze intenzifikacije kako bi kontinuirano poboljšavao rešenja tokom evolucije.

1. Lokalna pretraga (Local Search)

Ova faza koristi princip „dodaj ili ukloni čvor” kako bi unapredila trenutno najbolje rešenje:

- Ako je rešenje validno ($\text{fitness}(\mathbf{x}) \geq 1$), pokušava se uklanjanje čvora sa verovatnoćom obrnuto proporcionalnom njegovom stepenu.
- Ako nije validno, dodaje se čvor sa verovatnoćom proporcionalnom stepenu.

Ovaj princip postiže balans između *eksploracije* i *eksploatacije*.

2. Filtriranje (Filtering)

Kada je pronađen validan dominirajući skup, filtriranje pokušava da ga dodatno smanji uklanjanjem redundantnih čvorova (onih čije uklanjanje ne narušava pokrivenost). Proces se ponavlja dok postoji poboljšanje fitness funkcije.

3. Inspiracija elitnih rešenja (Elite Dominating Sets Inspiration)

Na kraju evolucije čuva se lista n_{DS} najboljih rešenja (*elite sets*). Od njih se formira tzv. jezgro ($\mathbf{x}_{\text{core}}$) kao presek n_{core} najboljih jedinki. Ako jezgro nije validno, dodaju se čvorovi najvećeg stepena dok ne postane validno.

Pseudokod HGA-MDS

Algorithm 7 HGA-MDS sa intenzifikacionim fazama

- 1: Inicijalizuj populaciju P_0 veličine M
 - 2: Proceni fitness svake jedinke
 - 3: Primeni **lokalnu pretragu** na najbolje rešenje
 - 4: **for** svake generacije $t = 1$ do g_{max} **do**
 - 5: Selekcija roditelja pomoću *linear ranking* metode
 - 6: Ukrštanje (p_c)
 - 7: Mutacija (p_m)
 - 8: Zamena generacije i elitizam
 - 9: Primeni **lokalnu pretragu i filtriranje**
 - 10: Ažuriraj skup najboljih DS rešenja
 - 11: **end for**
 - 12: Primeni **Elite Inspiration** fazu
 - 13: **Vrati** globalno najbolje rešenje
-

Parametri algoritma

Parametar	Opis	Tipična vrednost	Izvor
M	Broj jedinki u populaciji	40	[1]
$g_{\max}$	Maksimalan broj generacija	100	[1]
p_c	Verovatnoća ukrštanja	0.8	Standardno
p_m	Verovatnoća mutacije	0.01	Standardno
n_l	Broj iteracija u lokalnoj pretrazi	2	[1]
n_{DS}	Broj čuvanih elitnih rešenja	10	[1]
n_{core}	Broj elitnih rešenja za inspiraciju	3	[1]
MAX	MAX vrednost u linear ranking selekciji	1.1	[2]

Prednosti i ograničenja

Prednosti	Ograničenja
Kombinuje globalnu (GA) i lokalnu pretragu (LocalSearch) - visoki kvalitet rešenja	Više parametara koje je potrebno fino podešavati
Rang-bazirana selekcija sprečava preranu konvergenciju	Veće vreme računanja od standardnog GA
Bitarray omogućava vrlo efikasnu manipulaciju genetskog koda	Zahteva dodatnu biblioteku (bitarray)
Faze filtriranja i elitne inspiracije dodatno smanjuju veličinu MDS-a	Veća složenost implementacije

8 Eksperimentalni rezultati i analiza performansi

Opis eksperimentalnog okruženja

Eksperimenti su sprovedeni nad skupom test grafa preuzetih iz javno dostupnog repozitorijuma: `MarioGrobler/ds_verifier/testset`. Za testiranje performansi algoritama korišćeni su grafovi različitih veličina (od 20 do 300 čvorova).

U eksperimentima je upoređeno ukupno sedam pristupa:

- Metod grube sile (Brute Force)
- Pohlepna pretraga (Greedy Search)
- Lokalna pretraga (Local Search)
- Simulirano kaljenje (Simulated Annealing)
- Tabu pretraga (Tabu Search)
- Genetski algoritam (GA)
- Hibridni genetski algoritam (HGA-MDS)

Parametri eksperimenta

Za algoritme koji zahtevaju podešavanje hiperparametara korišćene su sledeće vrednosti:

Algoritam	Ključni parametri
Brute Force	- (egzaktna metoda)
Pohlepna pretraga (Greedy)	- (bez parametara)
Lokalna pretraga	<code>max_iters = 1000, init = "greedy"</code>
Simulirano kaljenje	<code>max_iters = 1000, init = "greedy", $T_0 = 10$, $\alpha = 0.99995$</code>
Tabu pretraga	<code>max_iters = 1000, tabu_tenure = 20</code>
Genetski algoritam (GA)	<code>population_size = 350, tournament_size = 5, mutation_rate = 0.03, elitism_size = 12, max_iters = 1000</code>
HGA-MDS	<code>M = 40, g_max = 100, p_c = 0.8, p_m = 0.01, n_l = 2, n_DS = 10, n_core = 3, max_rank_value = 1.1</code>

Tabela 10: Ključni parametri korišćenih algoritama.

Svi eksperimenti su izvedeni na istom hardverskom okruženju.

Ograničenja metode grube sile

Metod grube sile (Brute Force) korišćen je samo za najmanje grafove (do 20 čvorova), jer njegova složenost raste eksponencijalno:

$$O(2^{|V|} \cdot (|V| + |E|))$$

Za grafove veće od 20 čvorova izvršavanje je postalo vremenski neizvodljivo (prekinuto nakon nekoliko minuta). Ipak, ovaj pristup je poslužio kao referenca za tačna (optimalna) rešenja kod malih instanci.

Eksperimentalni rezultati

Tabela 11 prikazuje prosečna vremena izvršavanja (u milisekundama) za grafove veličine 20–300 čvorova.

Broj čvorova	Brute Force	Greedy	Local Search	Sim. Annealing	Tabu Search	GA	HGA-MDS
20	39749	<10	14	16	11	10136	133
50	—	<10	19	24	47	12607	186
100	—	10	27	35	71	14675	359
150	—	10	38	48	92	18229	423
200	—	12	46	59	125	21684	547
250	—	15	60	70	166	27927	642
300	—	18	71	87	194	28335	636

Tabela 11: Prosečno vreme izvršavanja algoritama (u milisekundama).

Rezultati veličine minimalnog dominirajućeg skupa prikazani su u tabeli 12.

Broj čvorova	Brute Force	Greedy	Local Search	Sim. Annealing	Tabu Search	GA	HGA-MDS
20	9	10	9	9	9	9	10
50	—	21	18	19	18	18	19
100	—	33	31	32	30	29	36
150	—	48	44	40	42	43	51
200	—	66	61	61	58	56	68
250	—	87	83	83	77	77	90
300	—	96	96	92	89	84	104

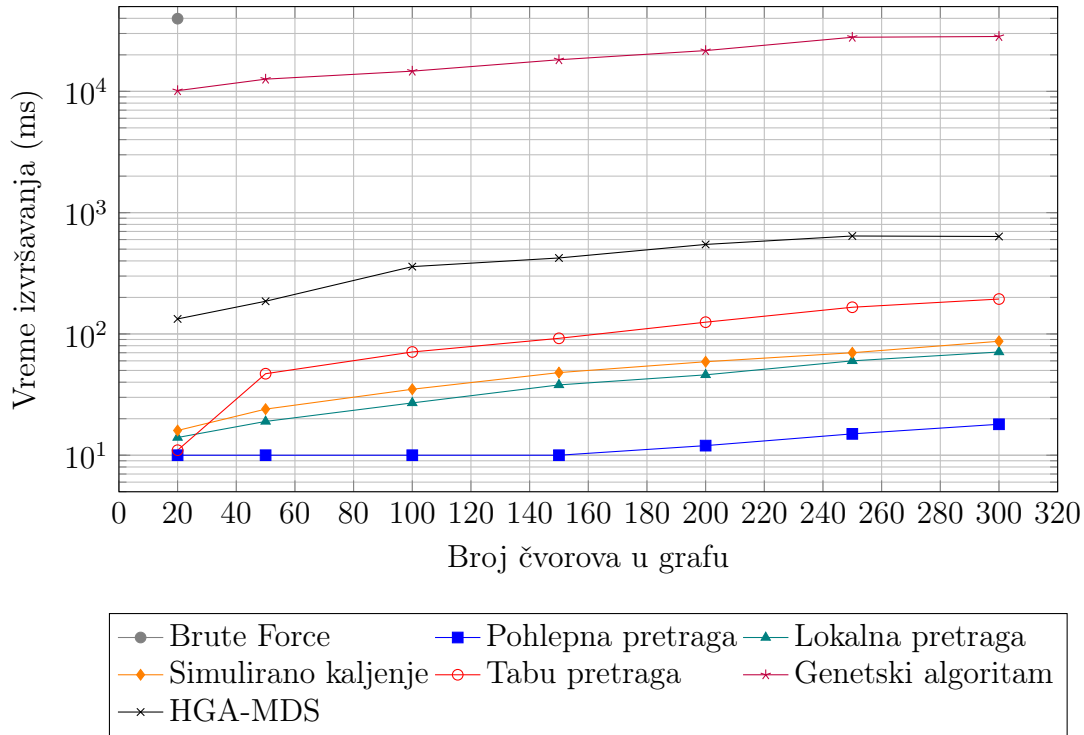
Tabela 12: Veličine pronađenih minimalnih dominirajućih skupova (MDS).

Analiza rezultata

Analiza vremena izvršavanja

- **Brute Force** algoritam je ekstremno spor (približno 40s već kod 20 čvorova), jer proverava sve podskupove čvorova. Korišćen je samo kao referenca za mala testiranja.
- **Greedy pristup** je najbrži - radi praktično trenutno za sve testirane grafove (<20 ms). Njegova složenost je približno $O(V^2)$.
- **Lokalna pretraga** i **Simulirano kaljenje** imaju slične performanse, linearno rastu sa brojem čvorova, dostižući 60–80 ms kod 300 čvorova.
- **Tabu pretraga** je nešto sporija zbog evaluacije većeg broja suseda i održavanja tabu liste.
- **Genetski algoritam (GA)** ima značajno veće vreme izvršavanja (približno 10–28 s), što je posledica populacije od 350 jedinki i 1000 generacija.

Uporedna analiza vremena izvršavanja algoritama (logaritamska skala)

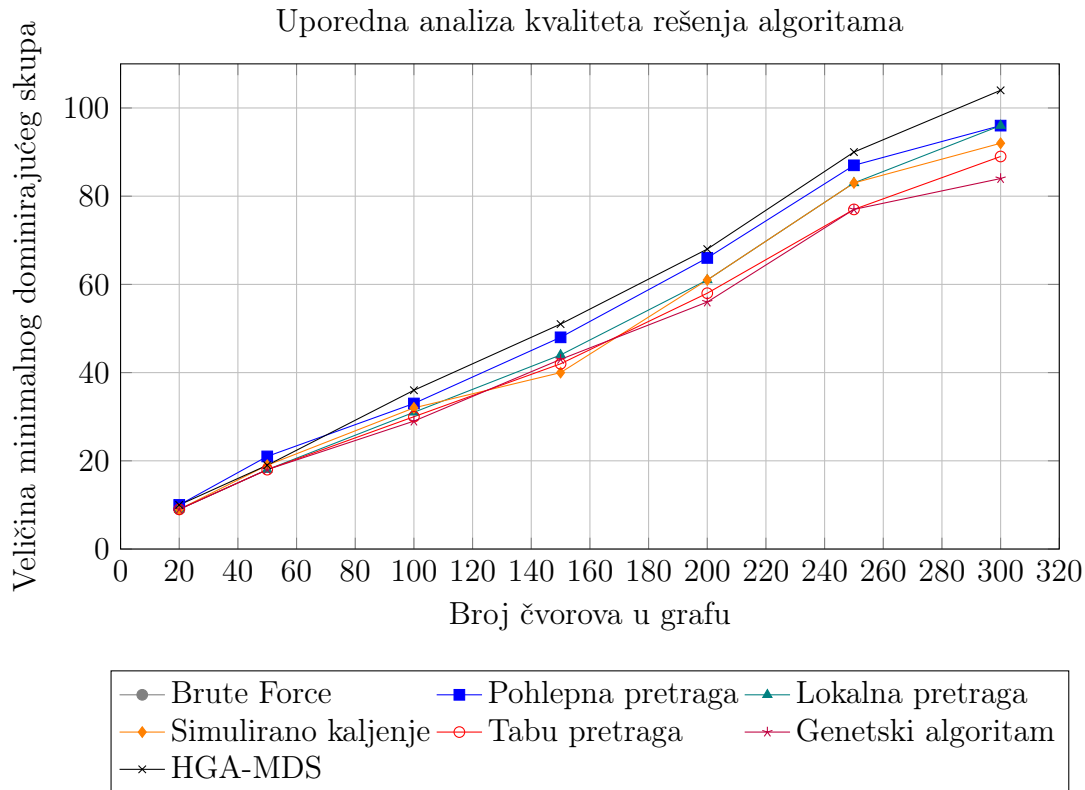


Slika 3: Uporedna analiza vremena izvršavanja algoritama za problem minimalnog dominirajućeg skupa (logaritamska osa).

- **HGA-MDS** je znatno efikasniji od standardnog GA - zahvaljujući manjoj populaciji (40) i kraćem broju generacija (100), prosečno vreme se smanjuje na ispod 1 s (500–650 ms).

Zaključak na osnovu vremena izvršavanja

Heuristike i metaheuristike (*Greedy*, *Local Search*, *Simulated Annealing*, *Tabu Search*) su brze i pogodne za veće grafove, dok *Genetski algoritam (GA)* i *HGA-MDS* zahtevaju znatno više vremena, ali daju kvalitetnija rešenja uz odgovarajuće podešavanje parametara.



Slika 4: Uporedna analiza kvaliteta rešenja algoritama za problem minimalnog dominirajućeg skupa.

Analiza kvaliteta rešenja

- **Brute Force** daje optimalno rešenje ($MDS = 9$ za $n = 20$) i služi kao referenca.
- **Greedy** proizvodi validne, ali suboptimalne rezultate - prosečno 10 - 25% veće od optimalnih.
- **Lokalna pretraga** i **Simulirano kaljenje** smanjuju veličinu skupa u odnosu na Greedy, naročito za male i srednje grafove (do 150 čvorova).
- **Tabu pretraga** daje još kompaktnije skupove, zahvaljujući pamćenju prethodnih poteza i izbegavanju lokalnih minimuma.
- **Genetski algoritam (GA)** postiže najmanje skupove za većinu grafova; na primer, za $n = 100$ pronalazi skup od 29 čvorova, što je najbolje među svim

metodama.

- **HGA-MDS** pokazuje blago veći broj čvorova u skupu (npr. 68 umesto 56 za $n = 200$), ali uz stabilniji proces konvergencije i konzistentnije rezultate između ponavljanja eksperimenta. Ovo je očekivano jer HGA-MDS koristi dodatne faze intenzifikacije (*LocalSearch*, *Filtering*, *Elite Inspiration*) i održava raznovrsnost populacije, što u nekim slučajevima daje nešto veće, ali stabilnije skupove.

Zaključak na osnovu kvaliteta

Najmanje dominirajuće skupove pronalaze **genetski algoritam** i **tabu pretraga**.

Zaključci eksperimentalne analize

1. **Brute Force** metoda potvrđuje tačnost implementacija na malim instancama, ali je vremenski neupotrebljiva već za $n > 20$.
2. **Pohlepna pretraga (Greedy)** je najbrža, ali daje neoptimalna rešenja - korisna je kao inicijalna aproksimacija.
3. **Lokalna pretraga** i **Simulirano kaljenje** ostvaruju vrlo dobar kompromis između tačnosti i brzine, naročito kod grafova do 200 čvorova.
4. **Tabu pretraga** se pokazuje kao najefikasniji balans - brzo konvergira i daje visokokvalitetna rešenja, često bliska rezultatima genetskog algoritma.
5. **Genetski algoritam (GA)** pronalazi najmanje MDS skupove, ali uz vrlo veliku vremensku cenu.
6. **HGA-MDS** značajno smanjuje vreme izvršavanja u odnosu na standardni GA (čak 40–50× brži), uz nešto slabiji ali stabilniji kvalitet rešenja što ga čini najpraktičnijim metaheurističkim pristupom za veće grafove.

Literatura

- [1] A.-R. Hedar and R. Ismail, *A Hybrid Genetic Algorithm for the Minimum Dominating Set Problem*, 2010.
- [2] J. E. Baker, *Adaptive Selection Methods for Genetic Algorithms*, Proceedings of the First International Conference on Genetic Algorithms, 1985.

- [3] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi, *Optimization by Simulated Annealing*, Science, 1983.
- [4] F. Glover, *Tabu Search - Part I*, ORSA Journal on Computing, 1989.
- [5] M. Živković, *Algoritmi*, Matematički fakultet, Beograd, 2015.